

UNIVERSIDAD DE INVESTIGACIÓN Y DESARROLLO – UDI

Maestría en Ciencia de Datos e Inteligencia Artificial

Programación con IA · Actividad Final

Pipeline de inteligencia artificial en Python para el mantenimiento predictivo de una fresadora CNC: variables físicas, umbral por costo y despliegue interactivo

Tema: Automatización de procesos

Presentado por:

María Fernanda Rivera Sanclemente

C.C. 25285106

Tutor:

Julian Andres Ramirez Bautista

Código fuente: <https://github.com/mrivera7-max/programacion-ia-udi.git>

Cuaderno ejecutable:

<https://colab.research.google.com/drive/1svM7ajSniB4S1ltB7fEzR6OMmsShFkpb#scrollTo=438a3969>

Bucaramanga, octubre de 2026

Contenido

Resumen	3
Abstract.....	3
1. Introducción	4
1.1 Planteamiento del problema	4
1.2 Hipótesis.....	4
1.3 Objetivos	4
1.4 Alcance y limitaciones	5
2. Marco conceptual.....	5
3. Metodología	5
3.1 Datos	6
3.2 Variables con fundamento físico	6
3.3 Protocolo experimental	7
3.4 Arquitectura orientada a objetos	7
3.5 Entorno y reproducibilidad	8
4. Desarrollo del taller	8
4.1 Refactorización asistida por IA: del script a la POO	8
4.2 Análisis exploratorio	9
4.3 Modelado predictivo adaptado a los datos.....	10
4.4 Dashboard interactivo.....	11
4.5 Simulador operativo con Gradio	12
4.6 Calidad del código y evidencia de ejecución.....	12
4.7 Atención a las observaciones del tutor	¡Error! Marcador no definido.
5. Resultados y discusión	13
5.1 Desempeño en el conjunto de prueba	13
5.2 Contraste de hipótesis	13
5.3 Análisis de errores e interpretación del dominio.....	15
5.4 Amenazas a la validez	15
6. Conclusiones	15
6.1 Importancia y aplicación desde mi perfil profesional	16
6.2 Trabajo futuro	16
Referencias.....	17
Anexo A. Bitácora de IA asistida	18

Resumen

Se desarrolló un pipeline de inteligencia artificial en Python, orientado a objetos y asistido por IA generativa, para predecir fallas de una fresadora CNC con el dataset AI4I 2020 (10 000 ciclos; 3,39 % de fallas). La metodología siguió CRISP-DM y contrastó dos hipótesis: que las variables derivadas de principios físicos mejoran el desempeño y que un umbral de decisión basado en costos reduce el costo de mantenimiento frente al umbral por defecto. Se compararon regresión logística, Random Forest y Gradient Boosting con validación cruzada estratificada de cinco pliegues, sin usar el conjunto de prueba en ninguna decisión.

Random Forest obtuvo en prueba una PR-AUC de 0,878 (IC 95 % bootstrap: 0,803–0,939), recall de 83,8 % y precisión de 90,5 %, con 57 de 68 fallas detectadas y 6 falsas alarmas. El costo esperado bajó 83 % frente a no predecir. Las variables físicas (ΔT , potencia y desgaste-torque) elevaron la PR-AUC de 0,736 a 0,891, con mejora en los cinco pliegues. El análisis de errores mostró que 8 de las 11 fallas no detectadas se deben a desgaste aleatorio de herramienta, lo que justifica una estrategia híbrida predictiva-preventiva. Los resultados se comunican en un dashboard Plotly y un simulador Gradio.

Palabras clave: *mantenimiento predictivo, programación orientada a objetos, aprendizaje automático, clases desbalanceadas, umbral por costo, IA asistida, Industria 4.0.*

Abstract

An object-oriented, AI-assisted Python pipeline was developed to predict failures of a CNC milling machine using the AI4I 2020 dataset (10,000 cycles; 3.39 % failures). Following CRISP-DM, two hypotheses were tested: physics-based features improve performance, and a cost-based decision threshold lowers maintenance cost relative to the default threshold. On the held-out test set, Random Forest reached a PR-AUC of 0.878 (95 % bootstrap CI: 0.803–0.939), 83.8 % recall and 90.5 % precision, cutting expected cost by 83 %. Physics-based features raised cross-validated PR-AUC from 0.736 to 0.891, improving in all five folds. Most missed failures stem from random tool wear, supporting a hybrid predictive-preventive strategy.

Keywords: *predictive maintenance, object-oriented programming, machine learning, class imbalance, cost-sensitive threshold, AI-assisted programming.*

1. Introducción

En manufactura, una parada no programada detiene la línea, daña la pieza y desgasta la herramienta. El mantenimiento correctivo llega tarde y el preventivo por calendario reemplaza componentes que aún sirven. El mantenimiento predictivo (PdM) usa datos de operación para intervenir solo cuando el riesgo lo justifica (Mobley, 2002), y es un componente central de la automatización de procesos en la Industria 4.0.

Este trabajo construye un pipeline de IA completo, desde la carga de datos hasta un dashboard de decisión, para una fresadora CNC. El proyecto integra los saberes de las unidades 1 a 3 del curso: programación orientada a objetos y patrones de diseño, IA asistida mediante prompts y modelado predictivo con visualización.

1.1 Planteamiento del problema

El dataset presenta tres retos que invalidan un enfoque ingenuo. Primero, la clase positiva es rara (3,39 %): un modelo que nunca alarme alcanza 96,6 % de exactitud. Segundo, los errores tienen costos asimétricos: una falla no detectada cuesta más que una inspección innecesaria. Tercero, las columnas de modo de falla (TWF a RNF) describen la falla ya ocurrida, por lo que usarlas como entrada produce fuga de información.

Pregunta de investigación. ¿Con qué desempeño y a qué costo se pueden predecir las fallas de una fresadora CNC a partir de seis variables de operación, y qué aportan el conocimiento físico del proceso y un umbral basado en costos?

1.2 Hipótesis

- **H1.** Agregar variables derivadas de principios físicos (ΔT , potencia mecánica y desgaste·torque) aumenta la PR-AUC en validación cruzada frente a usar solo las variables crudas.
- **H2.** Un umbral elegido para minimizar el costo esperado ($FN = 10, FP = 1$) produce menor costo en prueba que el umbral por defecto de 0,50.

1.3 Objetivos

Objetivo general. Desarrollar un pipeline reproducible, orientado a objetos y asistido por IA que entrene, evalúe y despliegue un modelo predictivo de fallas con un dashboard interactivo para la toma de decisiones.

Objetivos específicos.

1. Validar y preparar el dataset AI4I 2020 evitando la fuga de información.
2. Derivar variables con fundamento físico y medir su aporte con un estudio de ablación (H1).
3. Comparar tres algoritmos con validación cruzada estratificada y elegir el umbral por costo mínimo (H2).

4. Refactorizar el código a POO con los patrones Strategy, Factory y Facade, documentando el uso de IA asistida.
5. Comunicar los resultados en un dashboard interactivo y un simulador operativo.

1.4 Alcance y limitaciones

El dataset es sintético, aunque reproduce mecanismos de falla documentados (Matzka, 2020b). Los resultados validan el método, no una máquina concreta. Los costos $FN = 10$ y $FP = 1$ son supuestos explícitos y parametrizables.

2. Marco conceptual

Mantenimiento predictivo. Estima la probabilidad de falla a partir del estado de la máquina para programar la intervención antes de que ocurra (Mobley, 2002). Su valor depende de equilibrar fallas evitadas contra inspecciones adicionales.

Evaluación con clases desbalanceadas. Cuando la clase positiva es rara, la exactitud y la ROC-AUC sobrestiman el desempeño. La curva precisión-recall y su área (PR-AUC) son más informativas porque se centran en la clase minoritaria (Saito y Rehmsmeier, 2015).

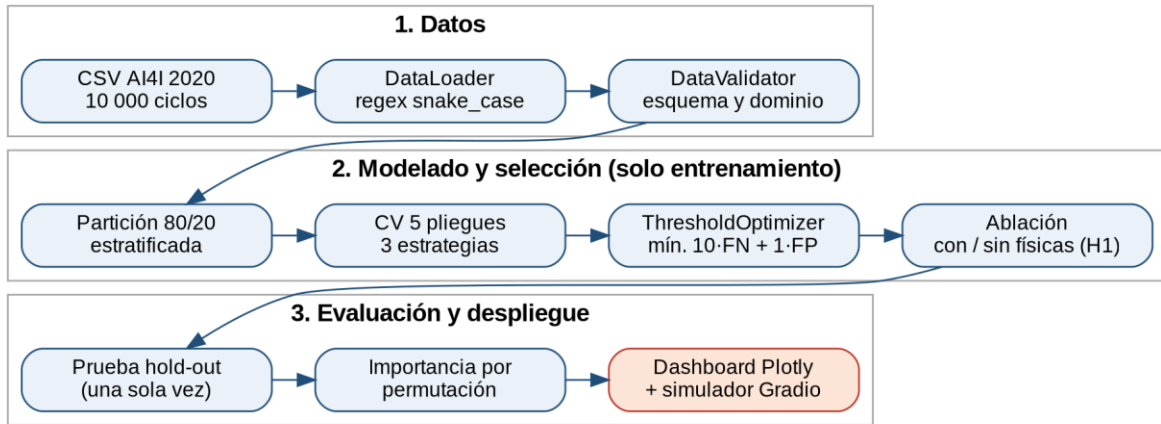
Decisión sensible al costo. Un clasificador probabilístico solo produce probabilidades; la decisión depende del umbral. Si un falso negativo cuesta $c(FN)$ y un falso positivo $c(FP)$, el umbral óptimo minimiza $c(FN) \cdot FN + c(FP) \cdot FP$ y, con costos asimétricos, queda por debajo de 0,50.

Programación orientada a objetos y patrones de diseño. El patrón Strategy encapsula algoritmos intercambiables tras una interfaz común; Factory centraliza su creación; Facade ofrece un punto de entrada único a un subsistema complejo (Gamma et al., 1994). Aplicados a un pipeline de ML, permiten agregar modelos sin modificar el orquestador (principio abierto/cerrado).

IA asistida en programación. Los modelos de lenguaje generan, refactorizan y documentan código a partir de prompts. Su uso responsable exige verificar cada salida con pruebas y criterio experto, porque el modelo puede proponer código plausible pero incorrecto.

3. Metodología

El estudio es cuantitativo, de tipo experimental computacional, y sigue las seis fases de CRISP-DM (Wirth y Hipp, 2000). La IA generativa actuó como asistente en cada fase, con validación humana antes de aceptar cada salida (Anexo A).

Figura 1*Flujo del pipeline de mantenimiento predictivo*

Nota. Elaboración propia. Las etapas 1 y 2 usan solo datos de entrenamiento; el conjunto de prueba se evalúa una única vez en la etapa 3.

3.1 Datos

Se usó el AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020a) del repositorio UCI. Contiene 10 000 ciclos de una fresadora y 14 columnas, no tiene valores nulos y registra 339 fallas (3,39 %).

Tabla 1*Variables del dataset y su rol en el modelo*

Variable	Unidad	Rol
type (L, M, H)	categoría	Entrada: calidad del producto
air_temperature_k	K	Entrada
process_temperature_k	K	Entrada
rotational_speed_rpm	rpm	Entrada
torque_nm	N·m	Entrada
tool_wear_min	min	Entrada
machine_failure	0/1	Variable objetivo
TWF, HDF, PWF, OSF, RNF	0/1	Excluidas: desglose de la falla (fuga de información)
UDI, product_id	ID	Excluidas: identificadores

3.2 Variables con fundamento físico

Las tres variables derivadas responden a los mecanismos de falla del proceso (Matzka, 2020b) y se calculan dentro del pipeline:

$$\Delta T = T_{\text{proceso}} - T_{\text{aire}} \text{ [K]} \quad (1)$$

$$P = T \cdot \omega = T \cdot 2\pi n / 60 \text{ [W]} \quad (2)$$

$$E = \text{desgaste} \cdot T \text{ [N}\cdot\text{m}\cdot\text{min]} \quad (3)$$

ΔT (ecuación 1) se asocia con la falla por disipación de calor; la potencia P (ecuación 2), con la falla por potencia; y E (ecuación 3), con la falla por sobreesfuerzo de la herramienta.

3.3 Protocolo experimental

1. **Partición:** 80/20 estratificada (8000 entrenamiento, 2000 prueba), semilla 42.
2. **Selección de modelo:** validación cruzada estratificada de 5 pliegues sobre entrenamiento; criterio, PR-AUC.
3. **Umbral:** barrido de 0,01 a 0,99 sobre las probabilidades fuera de pliegue, minimizando $10 \cdot FN + 1 \cdot FP$.
4. **H1 (ablación):** mismo algoritmo y mismos pliegues, con y sin variables físicas; comparación pareada por pliegue.
5. **Prueba:** evaluación única del modelo elegido con el umbral fijado; intervalos de confianza al 95 % por bootstrap de 2000 remuestreos (Efron y Tibshirani, 1993).
6. **Interpretación:** importancia por permutación medida como caída de PR-AUC en prueba.

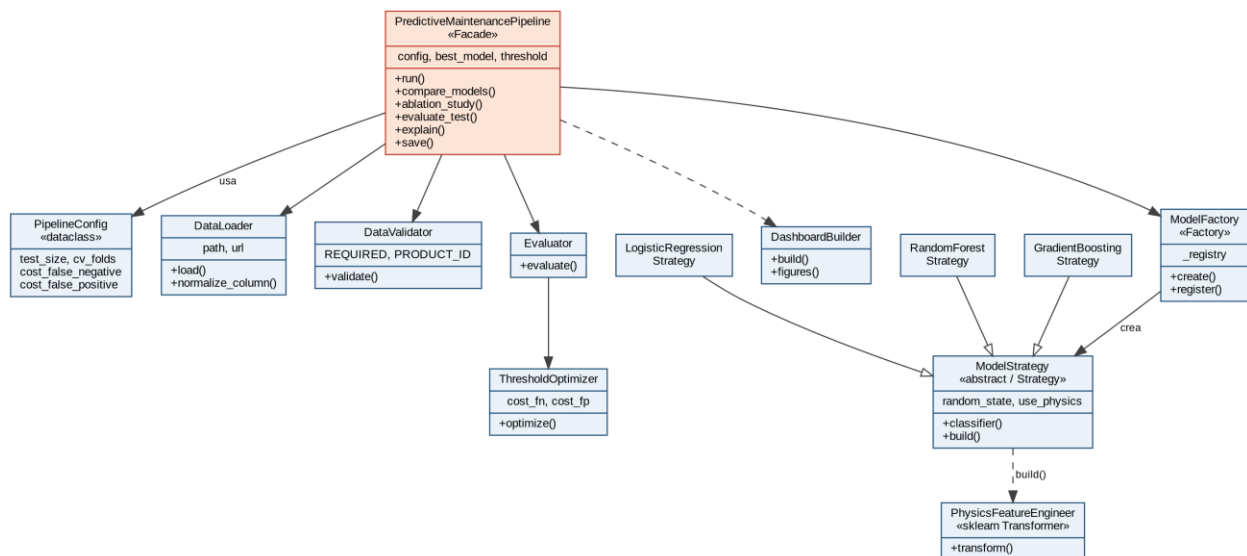
El escalado, la codificación one-hot y las variables físicas viven dentro de un `sklearn.Pipeline` (Pedregosa et al., 2011). Así se ajustan solo con los datos de entrenamiento de cada pliegue y se aplican idénticos en producción.

3.4 Arquitectura orientada a objetos

El código se organizó en el paquete `pdm_pipeline`, con siete módulos funcionales y catorce clases. Cada clase tiene una responsabilidad única (Figura 2 y Tabla 2).

Figura 2

Diagrama de clases del paquete `pdm_pipeline`



Nota. Elaboración propia. Las flechas con punta hueca indican herencia; las discontinuas, dependencias de uso.

Tabla 2*Módulos, clases y patrones de diseño*

Módulo	Clases	Responsabilidad	Patrón
config.py	PipelineConfig	Parámetros inmutables y reproducibles	Dataclass
data.py	DataLoader, DataValidator	Carga, normalización con regex y validación de esquema	Fail fast
features.py	PhysicsFeatureEngineer	Variables físicas como transformador de scikit-learn	Transformer
models.py	ModelStrategy, 3 estrategias, ModelFactory	Algoritmos intercambiables y su creación	Strategy + Factory
evaluation.py	Evaluator, ThresholdOptimizer, EvaluationReport	Métricas y umbral por costo	Value object
pipeline.py	PredictiveMaintenancePipeline	Orquesta las etapas con logging	Facade
dashboard.py, static_figures.py	DashboardBuilder, StaticFigureExporter	Dashboard interactivo y figuras del informe	Builder

3.5 Entorno y reproducibilidad

Tabla 3*Entorno de ejecución*

Elemento	Detalle
Lenguaje y bibliotecas	Python 3, scikit-learn, pandas, NumPy, Plotly, Gradio, joblib, pytest
Plataformas	Google Colab (cuaderno ejecutado de principio a fin) y entorno local
Semilla	42 en partición, validación cruzada, modelos y bootstrap
Ejecución	PYTHONPATH=src python -m pdm_pipeline o el cuaderno de Colab
Salidas	modelo_pdm.joblib, metricas.json, predicciones_prueba.csv, dashboard.html, figuras PNG

4. Desarrollo del taller

4.1 Refactorización asistida por IA: del script a la POO

La versión 0 (legacy/script_secuencial.py) era un script de 15 líneas que entrenaba un Random Forest y reportaba 98,5 % de exactitud. Con el prompt P3 (Anexo A) se refactorizó a POO; la Tabla 4 resume los cambios.

Tabla 4*Comparación entre el script inicial y el pipeline refactorizado*

Aspecto	Versión 0 (secuencial)	Versión final (POO)
Estructura	Código global, sin funciones	7 módulos, 14 clases, patrones Strategy, Factory y Facade
Validación de datos	Ninguna	Esquema, nulos, dominio y formato de ID con regex
Fuga de información	No controlada	7 columnas excluidas; preprocesamiento dentro del Pipeline
Partición	Aleatoria, sin semilla	Estratificada y reproducible (semilla 42)
Métrica	Exactitud (98,5 %, engañosa)	PR-AUC, recall, precisión y costo esperado
Umbral	0,50 implícito	Óptimo por costo con datos fuera de pliegue
Documentación y calidad	Sin docstrings ni pruebas	Docstrings Google, PEP 8 (flake8), 5 pruebas pytest, logging
Comunicación	Un print	Dashboard Plotly (8 figuras) y simulador Gradio

El siguiente fragmento muestra el patrón Factory. Agregar un algoritmo nuevo solo requiere registrar una clase, sin modificar el orquestador:

```
class ModelFactory:
    """Fábrica que traduce un nombre en una estrategia concreta."""

    _registry: dict[str, type[ModelStrategy]] = {
        cls.name: cls for cls in (LogisticRegressionStrategy,
                                   RandomForestStrategy,
                                   GradientBoostingStrategy)
    }

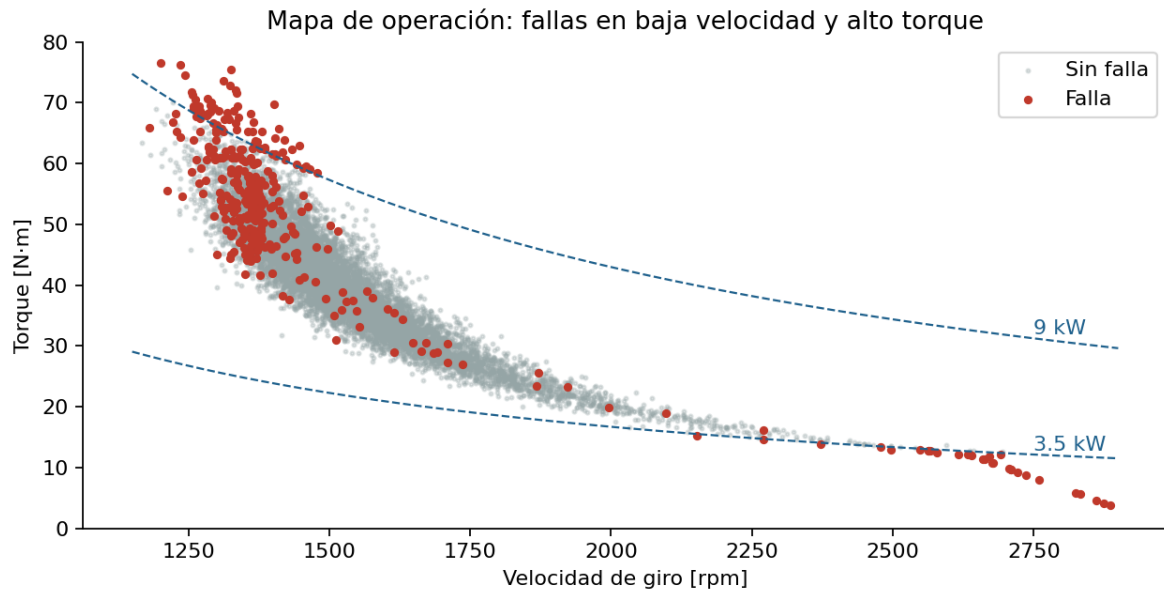
    @classmethod
    def create(cls, name: str, random_state: int = 42,
               use_physics: bool = True) -> ModelStrategy:
        """Instancia la estrategia registrada con ``name``."""
        if name not in cls._registry:
            raise KeyError(f"Modelo '{name}' no registrado")
        return cls._registry[name](random_state=random_state,
                                    use_physics=use_physics)
```

4.2 Análisis exploratorio

Las fallas se concentran en dos regiones del mapa de operación: baja velocidad con alto torque, y alta velocidad con torque muy bajo, fuera de la banda de potencia de 3,5 a 9 kW (Figura 3).

Figura 3

Mapa de operación: torque frente a velocidad de giro, coloreado por falla

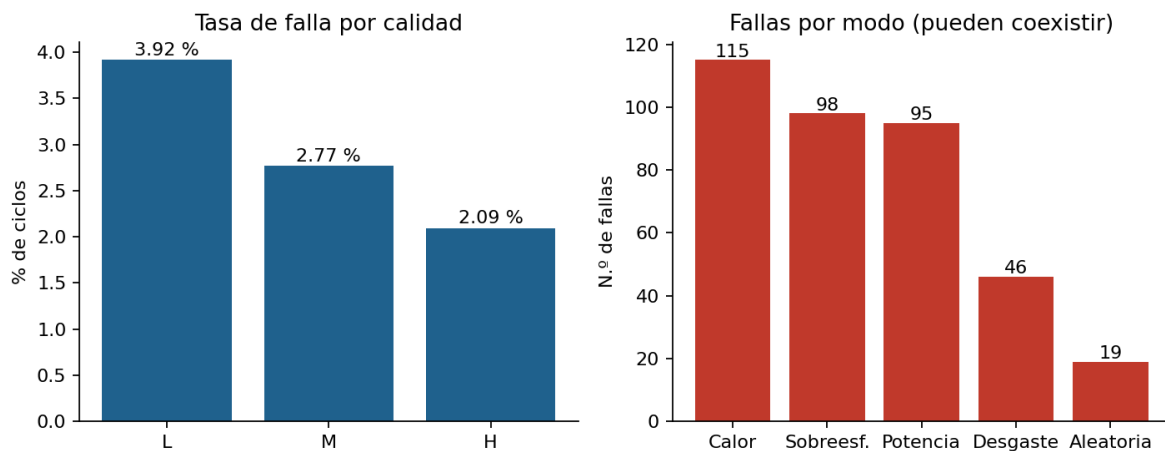


Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

La calidad L falla casi el doble que la H (3,92 % frente a 2,09 %). La disipación de calor es el modo más frecuente (115 casos) y el aleatorio el menos frecuente (19), impredecible por definición (Figura 4).

Figura 4

Tasa de falla por calidad del producto y número de fallas por modo



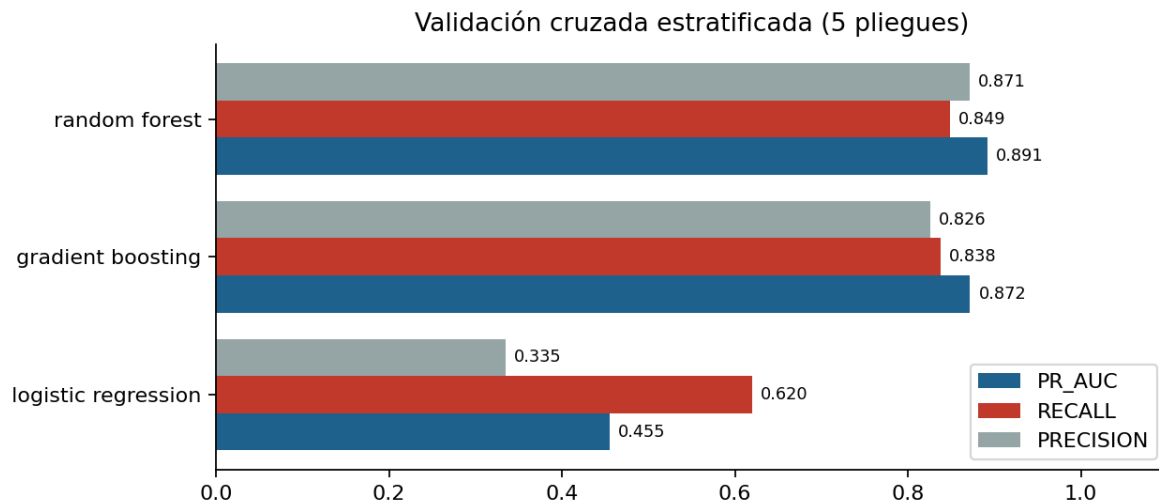
Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

4.3 Modelado predictivo adaptado a los datos

Random Forest (Breiman, 2001) obtuvo la mayor PR-AUC en validación cruzada (0,891; desviación estándar entre pliegues de 0,030). La regresión logística quedó en 0,455 porque las fronteras de falla no son lineales (Figura 5 y Tabla 5).

Figura 5

Comparación de modelos en validación cruzada estratificada de 5 pliegues



Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

Tabla 5

Resultados de validación cruzada por modelo

Modelo	PR-AUC	DE entre pliegues	Recall	Precisión	Umbral	Costo
Random Forest	0,891	0,030	0,849	0,871	0,34	444
Gradient Boosting (HGB)	0,872	0,029	0,838	0,826	0,12	488
Regresión logística	0,455	0,047	0,620	0,335	0,80	1364

Nota. Métricas calculadas sobre las 8000 predicciones fuera de pliegue; DE: desviación estándar de la PR-AUC entre los 5 pliegues. $Costo = 10 \cdot FN + 1 \cdot FP$.

El umbral de cada modelo se fijó minimizando $10 \cdot FN + 1 \cdot FP$ sobre las predicciones fuera de pliegue. El conjunto de prueba no intervino en ninguna decisión de modelado.

4.4 Dashboard interactivo

El pipeline genera `outputs/dashboard.html`, un tablero Plotly autocontenido con seis indicadores clave y ocho figuras interactivas. Cada figura responde una pregunta concreta de la planta (Tabla 6).

Tabla 6

Componentes del dashboard y decisiones que apoyan

Panel	Pregunta que responde	Decisión que apoya
Indicadores (KPI)	¿Cuánto mejora el modelo frente a no predecir?	Justificar la inversión en PdM
Mapa de operación	¿En qué condiciones falla la máquina?	Definir ventanas de operación seguras

Panel	Pregunta que responde	Decisión que apoya
Fallas por calidad y modo	¿Qué productos y mecanismos dominan?	Priorizar controles por tipo de producto
Comparación de modelos	¿Qué algoritmo usar?	Seleccionar el modelo de producción
Matriz de confusión	¿Cuántas fallas se escapan y cuántas alarmas sobran?	Dimensionar el equipo de inspección
Costo vs. umbral	¿Desde qué probabilidad conviene alarmar?	Fijar la política de alarma
Importancia de variables	¿Qué sensor vigilar?	Priorizar instrumentación
Riesgo por desgaste	¿Cuándo cambiar la herramienta?	Definir la regla preventiva

4.5 Simulador operativo con Gradio

El módulo `app_gradio.py` despliega un simulador web en el que el operario ajusta calidad, temperaturas, velocidad, torque y desgaste. El simulador devuelve la probabilidad de falla, la decisión con el umbral de costo mínimo y las magnitudes físicas derivadas. Se validó con dos casos de control:

Tabla 7
Casos de validación del simulador

Caso	Condición de entrada	Riesgo	Decisión
Crítico	L, 302 K / 311 K, 1300 rpm, 65 N·m, 215 min (8,85 kW)	95,9 %	Programar mantenimiento
Nominal	M, 300 K / 310 K, 1500 rpm, 40 N·m, 50 min (6,28 kW)	0,0 %	Operación normal

4.6 Calidad del código y evidencia de ejecución

El código cumple PEP 8 (Van Rossum et al., 2001), verificado con flake8 (línea máxima de 100 caracteres). Todas las clases y métodos públicos tienen docstrings estilo Google con secciones Args, Returns y Raises. Cinco pruebas unitarias con pytest cubren la normalización con regex, el cálculo de potencia, el validador, el optimizador de umbral y la fábrica de modelos.

El cuaderno de Colab se ejecutó de principio a fin sin errores. Reprodujo exactamente las métricas de este informe y las 5 pruebas aprobadas, y generó el enlace público del simulador.

Repositorio: https://github.com/mrivera7-max/programacion-ia-udi/tree/main/actividad_final/actividad_final

Cuaderno de Colab: [Actividad Final Pipeline IA.ipynb](#)

5. Resultados y discusión

5.1 Desempeño en el conjunto de prueba

En prueba (2000 ciclos, 68 fallas), el modelo detectó 57 fallas con 6 falsas alarmas. La PR-AUC en prueba (0,878) es coherente con la de validación (0,891) y no hay evidencia de sobreajuste.

Tabla 8

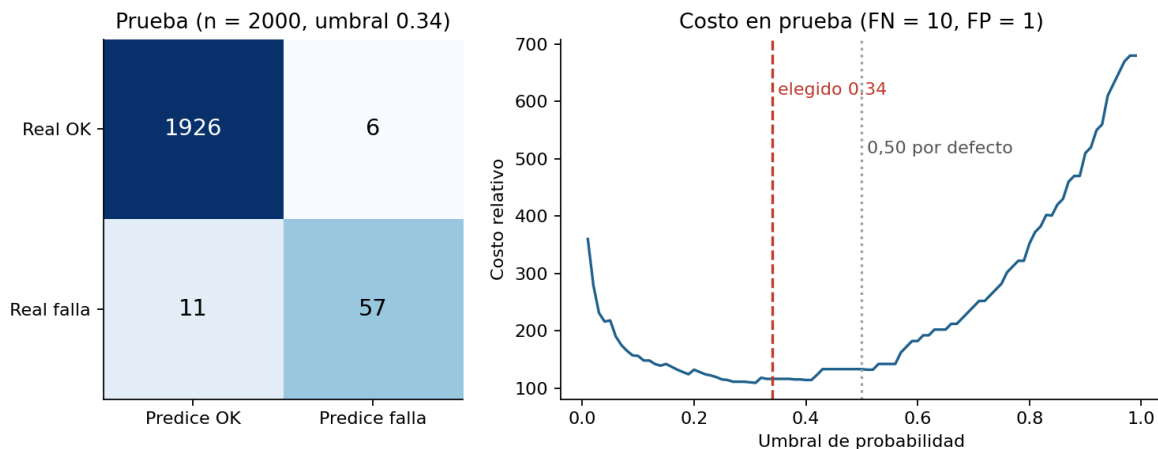
Métricas del modelo seleccionado en el conjunto de prueba

Métrica	Valor	IC 95 % (bootstrap)
PR-AUC	0,878	0,803 – 0,939
ROC-AUC	0,983	—
Recall (fallas detectadas)	83,8 % (57 de 68)	74,2 % – 92,2 %
Precisión (alarmas correctas)	90,5 % (57 de 63)	82,9 % – 97,0 %
F1	0,870	—
Costo esperado (FN = 10, FP = 1)	116 frente a 680 sin modelo (–83 %)	—

Nota. Intervalos por bootstrap percentil con 2000 remuestreos del conjunto de prueba (Efron y Tibshirani, 1993).

Figura 6

Matriz de confusión en prueba y costo según el umbral de decisión



Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

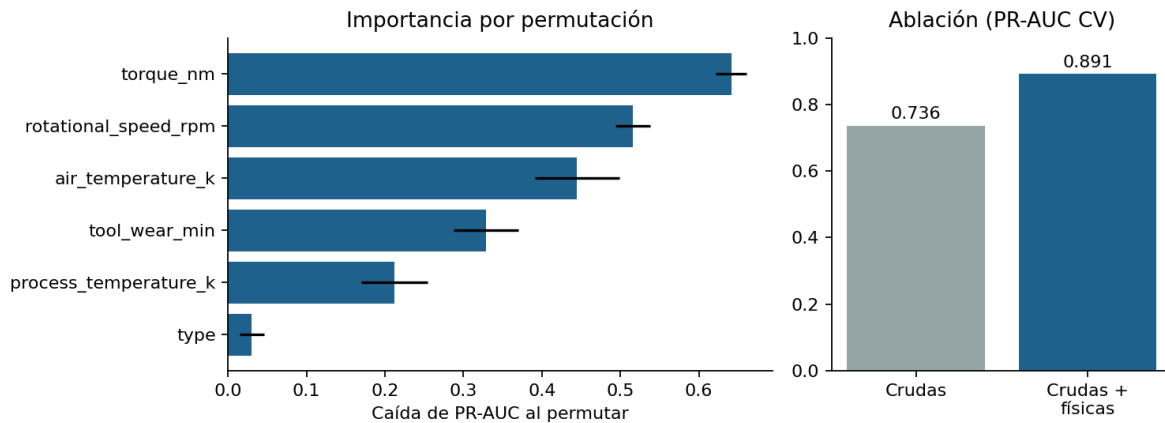
5.2 Contraste de hipótesis

H1 se acepta. Con el mismo algoritmo y los mismos pliegues, las variables físicas elevaron la PR-AUC de Random Forest de 0,736 a 0,891 (*medias por pliegue*: $0,740 \pm 0,034$ y $0,892 \pm 0,030$). La mejora se observó en los cinco *pliegues* ($+0,11$ a $+0,19$; *prueba t pareada*, $t(4) = 10,2$, $p < 0,001$), y también en Gradient Boosting (*medias por pliegue*: $0,792 \rightarrow 0,871$). El estadístico debe leerse con cautela, porque los pliegues comparten datos de entrenamiento y la

prueba t clásica subestima la varianza (Nadeau y Bengio, 2003). Aun así, que la mejora sea positiva en todos los pliegues es una evidencia robusta.

Figura 7

Importancia por permutación y estudio de ablación de las variables físicas



Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

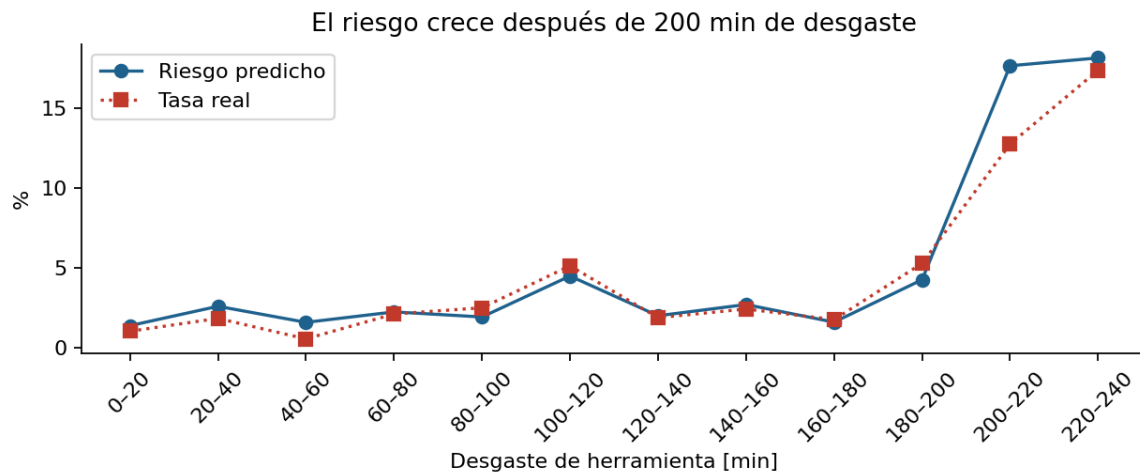
La mejora por conocimiento del dominio (+0,15) supera la diferencia entre los dos mejores algoritmos (+0,02). En este problema, entender la física del proceso aporta más que cambiar de algoritmo. El torque y la velocidad son las variables más influyentes, coherentes con el mecanismo de potencia.

H2 se acepta. Con el umbral por costo (0,34) el costo en prueba fue 116; con el umbral por defecto de 0,50 fue 133 (+15 %). La curva de costo es relativamente plana entre 0,20 y 0,45 (110 a 133), de modo que la política es robusta a errores moderados en la estimación de costos. En prueba, el umbral de 0,30 habría dado un costo algo menor (110), pero elegirlo con esos datos sería sobreajustar la decisión al conjunto de prueba.

5.3 Análisis de errores e interpretación del dominio

Figura 8

Riesgo de falla predicho y tasa real según el desgaste de la herramienta



Nota. Elaboración propia con el dataset AI4I 2020 (Matzka, 2020a).

De las 11 fallas no detectadas, 8 son por desgaste de herramienta (TWF). En este proceso la herramienta se reemplaza o falla en un instante aleatorio entre 200 y 240 min de uso (Matzka, 2020b), y la Figura 8 muestra el salto de riesgo a partir de 200 min. Ninguna variable de los sensores anticipa ese instante exacto: es un límite del problema, no del algoritmo.

La implicación operativa es una estrategia híbrida: el modelo predictivo vigila las fallas por calor, potencia y sobreesfuerzo, y una regla preventiva cambia la herramienta antes de 200 min de uso. El riesgo predicho sigue de cerca la tasa real en todos los tramos de desgaste, lo que indica una buena calibración del modelo.

5.4 Amenazas a la validez

- **Validez externa:** el dataset es sintético; en una planta real habrá ruido de medición, deriva de sensores y fallas no modeladas.
- **Validez de constructo:** la razón de costos 10:1 es un supuesto; el pipeline la recibe como parámetro para recalculer el umbral con costos reales.
- **Validez estadística:** con 68 fallas en prueba, los intervalos de recall son amplios (74 % a 92 %); conviene ampliar la muestra antes de un despliegue real.

6. Conclusiones

1. El pipeline responde la pregunta de investigación. Con seis variables de operación se detectó el 83,8 % de las fallas con 90,5 % de precisión, y el costo esperado bajó 83 % frente a no predecir.

2. El conocimiento físico supera al cambio de algoritmo (H1). Tres ecuaciones de ingeniería elevaron la PR-AUC de 0,736 a 0,891, con mejora en todos los pliegues. Esa ganancia (+0,155) es unas ocho veces la diferencia entre los dos mejores algoritmos (+0,019).
3. La métrica y el umbral son decisiones de ingeniería (H2). La exactitud de 98,5 % del script inicial ocultaba el problema. La PR-AUC y un umbral por costo alinean el modelo con la economía de la planta.
4. Declarar los límites del modelo es parte del resultado. La falla por desgaste ocurre en un instante aleatorio; se cubre mejor con una regla preventiva que con más datos de los mismos sensores.
5. La POO hace el pipeline mantenible y extensible. Strategy, Factory y Facade permiten agregar modelos, cambiar costos o sustituir el dashboard sin reescribir el orquestador.
6. La IA asistida acelera, pero no reemplaza el criterio. Los prompts agilizaron la refactorización, las pruebas y el dashboard. La validación humana (fuga de datos, unidades físicas, ablación, separación estricta de la prueba) fue la que hizo confiable el resultado.

6.1 Importancia y aplicación desde mi perfil profesional

Como ingeniera electrónica y docente del programa de Ingeniería Electrónica de la UDI, veo en este pipeline una herramienta aplicable en tres frentes:

- **Docencia:** un proyecto integrador que une instrumentación, control digital y ciencia de datos. Los estudiantes adquieren señales con microcontroladores y alimentan este mismo pipeline, conectando la física del sensor con la decisión basada en datos.
- **Investigación aplicada:** mantenimiento predictivo en el borde (edge AI). El modelo se ejecuta en un sistema embebido junto a la máquina, con sensores de corriente, vibración y temperatura.
- **Extensión con el sector productivo de Santander:** pilotos de bajo costo para talleres de mecanizado y plantas de alimentos de la región, donde las paradas no programadas tienen alto impacto.

6.2 Trabajo futuro

- Validar el pipeline con datos reales de una máquina, incorporando vibración y corriente del motor.
- Pasar de clasificar fallas a estimar la vida útil remanente (RUL) con modelos de series de tiempo.
- Cuantizar el modelo para microcontroladores (TinyML) y monitorear la deriva de datos en producción.
- Calibrar los costos FN y FP con información financiera real de la planta.

Referencias

- Anthropic. (2026). *Claude* [Modelo de lenguaje grande]. <https://claude.ai>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Efron, B. y Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Gradio. (s. f.). *Gradio Python client: Introduction*. Recuperado el 4 de octubre de 2026, de <https://gradio.app/docs/python-client/introduction>
- Matzka, S. (2020a). *AI4I 2020 predictive maintenance dataset* [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>
- Matzka, S. (2020b). Explainable artificial intelligence for predictive maintenance applications. En *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69–74). IEEE. <https://doi.org/10.1109/AI4I49448.2020.00023>
- Mobley, R. K. (2002). *An introduction to predictive maintenance* (2.^a ed.). Butterworth-Heinemann.
- Nadeau, C. y Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239–281. <https://doi.org/10.1023/A:1024068626366>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Plotly Technologies Inc. (2015). *Collaborative data science*. <https://plot.ly>
- Python Software Foundation. (s. f.-a). *logging — Logging facility for Python*. Recuperado el 4 de octubre de 2026, de <https://docs.python.org/3/library/logging.html>
- Python Software Foundation. (s. f.-b). *re — Regular expression operations*. Recuperado el 4 de octubre de 2026, de <https://docs.python.org/3/library/re.html>
- Saito, T. y Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Van Rossum, G., Warsaw, B. y Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. <https://peps.python.org/pep-0008/>

Wirth, R. y Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. En *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining* (pp. 29–39).

Anexo A. Bitácora de IA asistida

Seis prompts guiaron el desarrollo. Cada salida se aceptó solo después de la verificación indicada. La versión completa está en `prompts/bitacora_prompts.md` del repositorio.

Tabla 9

Prompts utilizados y verificación humana

ID	Etapas	Prompt (resumen)	Verificación humana
P1	Comprensión	«Actúa como científico de datos industrial. Enumera columnas con riesgo de fuga, efecto del desbalance y partición adecuada para AI4I 2020.»	Se confirmó que TWF–RNF desglosan la falla; se excluyeron 7 columnas.
P2	Variables físicas	«Soy ingeniera electrónica. Propón variables derivadas con ecuación y unidades para fallas por calor, potencia y sobreesfuerzo. No inventes umbrales.»	Prueba unitaria de P (40 N·m, 1500 rpm = 6283 W) y ablación 0,736 → 0,891.
P3	Refactorización	«Refactoriza este script a POO con SOLID; usa Strategy y Factory como en la Fase I; mantén compatibilidad con sklearn.Pipeline.»	Preprocesamiento dentro del Pipeline; misma salida que el script original.
P4	Calidad	«Revisa contra PEP 8, agrega docstrings estilo Google y escribe 5 pruebas pytest.»	flake8 sin advertencias; 5 pruebas aprobadas.
P5	Métrica y umbral	«Con 3,4 % de positivos, ¿por qué la exactitud de 0,985 engaña? Propón métrica y umbral si un FN cuesta 10 veces un FP.»	Umbral 0,34 elegido con datos fuera de pliegue, no con prueba.
P6	Dashboard	«Diseña un dashboard Plotly para un jefe de mantenimiento; cada gráfico responde una pregunta. Agrega un simulador Gradio.»	Alarma en el caso crítico (95,9 %); sin alarma en el nominal (0 %).